



دانشگاه صنعتی شریف

دانشکده مهندسی مکانیک

طراحی مسیر با الگوریتم دایکسترا

تمرین هفتم درس رباتیک

هانیه زهرا بوداگی - ۹۹۱۰۵۹۵۶

استاد

دکتر بهزادی پور

ترم اول سال ۱۴۰۲

فهرست مطالب

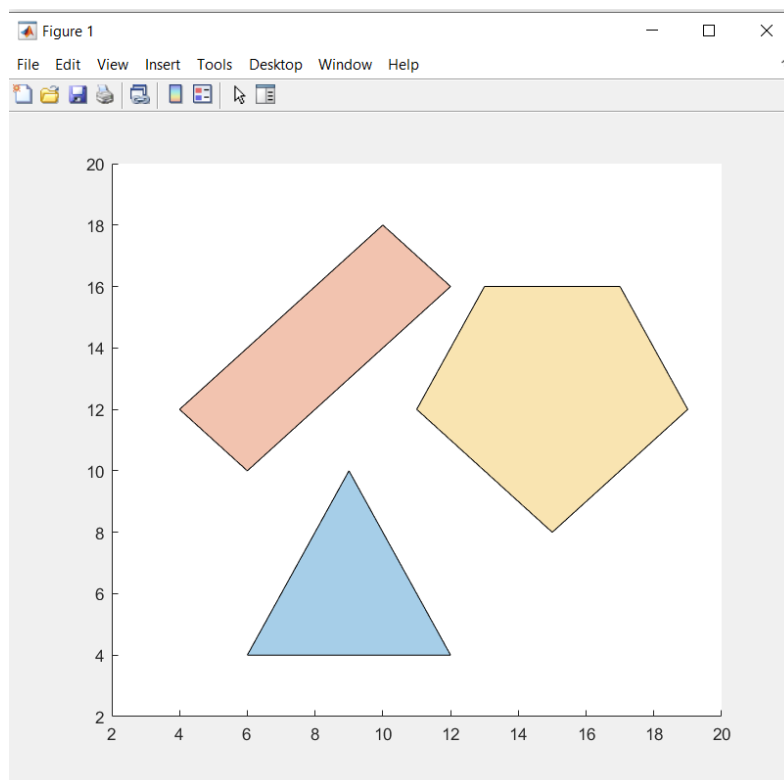
۱. صورت سوالات	1
۱.۱ سوال اول	1
۲. پاسخ سوالات	1
۱.۲ سوال اول	1

۱. صورت سوالات

در این مسئله از ما خواسته می‌شود تا با استفاده از الگوریتم دایکسترا، کوتاه‌ترین مسیر بین نقطه‌ی شروع و پایان را با وجود موانعی مشابه موانع تمرین قبل در صفحه بیابیم.

۱.۱ سوال اول

در اینجا ابتدا از ما خواسته شده تا اطلاعات موانع را که به صورت پاره خط داده شده‌اند، از یک فایل `.txt` بخوانیم و موانع را ترسیم کنیم. سپس با استفاده از الگوریتم دایکسترا، کوتاه‌ترین مسیر را یافته و رسم کنیم.



۲. پاسخ سوالات

۱.۲ سوال اول

کد مسئله را به شکل زیر در یک اسکریپت می‌زنیم (جزئیات کد قدم به قدم شرح داده خواهد شد):

```
clc; clear;

dataId = fopen('data1.txt', 'r');
data = fscanf(dataId, '%d', Inf);
fclose(dataId);

obst_no = data(1);
points_data = ones(obst_no, 4);
for i = 1:obst_no
```

```

    for j = 1:4
        points_data(i,j) = data(2+4*(i-1)+j-1);
    end
end

[sizeData,~] = size(data);
P_start = [data(sizeData-3); data(sizeData-2)];
P_end = [data(sizeData-1); data(sizeData)];

X = [points_data(1,1),points_data(1,3)];
Y = [points_data(1,2),points_data(1,4)];
points = [];
polylist = [];
N = 0;
flag = 0; %X does have a number within
for i = 1:obst_no-1
    if flag == 1
        X = [points_data(i+1,1),points_data(i+1,3)];
        Y = [points_data(i+1,2),points_data(i+1,4)];
        flag = 0;
        continue
    end

    if points_data(i,3) == points_data(i+1,1) && points_data(i,4) == points_data(i+1,2)
        X = [X,points_data(i+1,3)];
        Y = [Y,points_data(i+1,4)];
    end
    if X(end) == X(1) && Y(end) == Y(1)
        X = X(1:end-1);
        Y = Y(1:end-1);
        points = [points;X',Y'];
        N = N+1;
        pgon = polyshape(X,Y);
        polylist = [polylist,pgon];
        plot(pgon);
        hold on
        X = [];
        Y = [];
        flag = 1; %X is empty
    end
end

graph = graph_generator(P_start,P_end, points,polylist,N);
dijkstra(P_start',P_end',points,polylist,N);

```

در بخش اول این کد، تلاش می‌کنیم که داده‌های فایل `data.txt` را به صورت منظم و طبقه‌بندی شده درآورده و سپس موانع را از اطلاعات این فایل استخراج کنیم و به شکل چند ضلعی رسم نماییم.

در ادامه، به کشیدن گراف مربوط به روش نقشه‌ی راه (Road Mapping) با استفاده از تابع `graph_generator` می‌پردازیم که مسیر گراف را با رنگ قرمز روی پلات اولیه برای ما ترسیم می‌کند.

تابع و نتیجه‌ی بکارگیری آن به شرح زیر هستند:

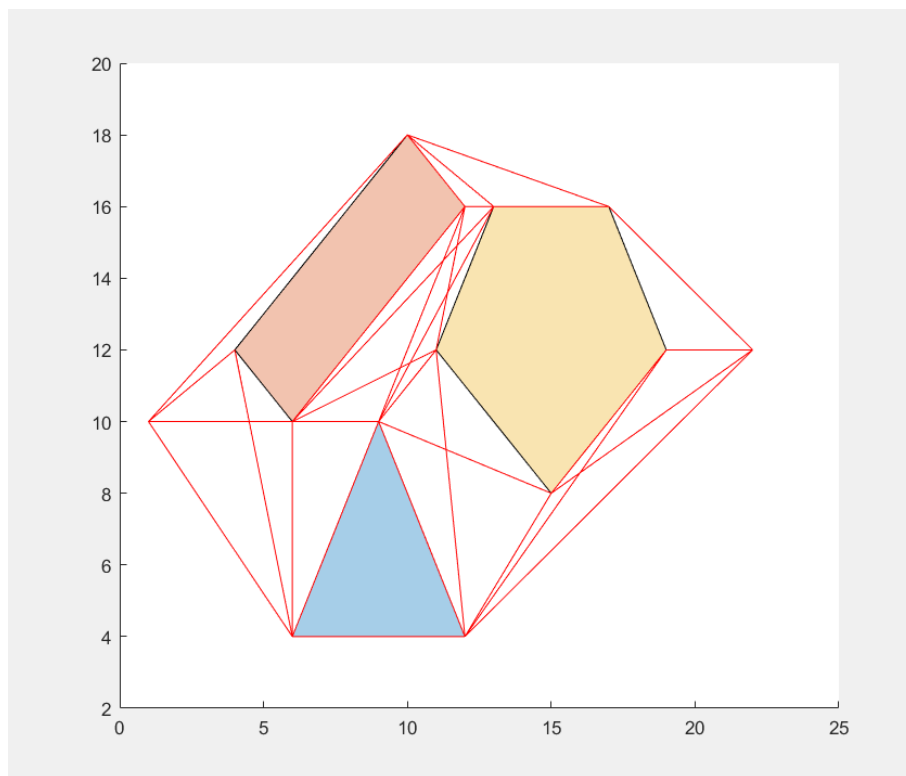
```

function graph = graph_generator(Ps,Pf,points,polylist,N)

allP = [Ps';points;Pf'];
s = size(allP);
number = s(1);
graph = [];
max_graph_lines = factorial(number)/(2*factorial(number-2));

for i = 1:max_graph_lines-1
    for j = 1:max_graph_lines+1
        if i+j <= number
            lineseg = [allP(i,:);allP(i+j,:)];
            IN = [];
            for k = 1:N
                [in,~] = intersect(polylist(k),lineseg);
                IN = [IN,in];
            end
            if isempty(IN)
                lineX = [lineseg(:,1)'];
                lineY = [lineseg(:,2)'];
                plot(lineX,lineY,'r');
                hold on
                graph = [graph; lineX, lineY];
            end
        end
    end
end
end
end

```



سپس، با استفاده از تابع `dijkstra`، مسیر نهایی تولید شده و با رنگ سبز روی پلات رسم می‌گردد.

کد تابع و نتیجه‌ی آن به شرح زیر است:

```
function dijkstra(Ps,Pf,points,polylist,N)

allPoints = flip([Ps;points;Pf]);
[number,~] = size(allPoints);

notMet = remove_row(allPoints,Pf);
X = Pf;
X_previous = [0,0];
Pdata = ones(number,3);
Pdata(1,1:2) = Pf; Pdata(1,3) = 0;
for j = 2:number
    Pdata(j,1:2) = allPoints(j,:);
    Pdata(j,3) = 1000;
end

while X(:) ~= Ps(:)
    neighbors = [];
    dis = [];
    n_neighbors = 0;
    [row,~] = size(notMet);
    for j = 1:row
        lineseg = [X;notMet(j,:)];
        IN = [];
        for k = 1:N
            [in,~] = intersect(polylist(k),lineseg);
            IN = [IN;in];
        end
        if isempty(IN)
            neighbors = [neighbors;notMet(j,:)];
            n_neighbors = n_neighbors + 1;
            dis = [dis; norm(X-notMet(j,:))];
        end
    end
    %find current node in Pdata
    for j = 1:number
        if X(1) == Pdata(j,1) && X(2) == Pdata(j,2)
            current = j;
            dis_current = Pdata(j,3);
        end
    end

    for j = 1:number
        for k = 1:n_neighbors
            if Pdata(j,1) == neighbors(k,1) && Pdata(j,2) == neighbors(k,2)
                dis(k) = norm(neighbors(k,:) - X) + dis_current;
                Pdata(j,3) = dis(k);
            end
        end
    end

    least_distant = 1;
```

```

least_distance = dis(1);

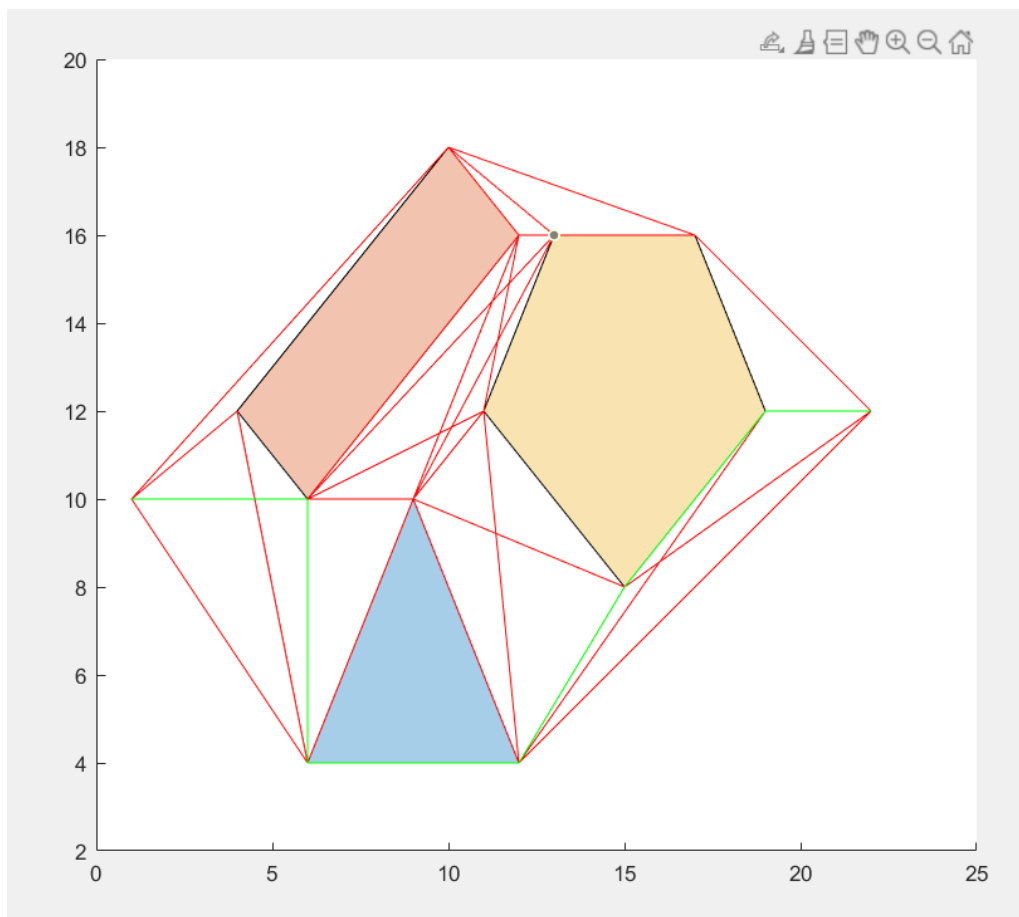
for j = 2:n_neighbors
    if dis(j) <= least_distance
        least_distance = dis(j);
        least_distant = j;
    end
end

pathX = [X(1), neighbors(least_distant,1)];
pathY = [X(2), neighbors(least_distant,2)];
plot(pathX,pathY,'g');
X_previous = X;
notMet = remove_row(notMet,X_previous);
X = neighbors(least_distant,:);

end

pathX = [X(1), Ps(1)];
pathY = [X(2), Ps(2)];
plot(pathX,pathY,'g');
end

```



در زدن این تابع، از تابع دیگری به نام `remove_row` نیز استفاده شده که برای حذف یک نقطه از یک دیتاست مشخص استفاده می‌شود.

کد این تابع:

```
function mat_modif = remove_row(matrix,row)
    [m,~] = size(matrix);
    for i = 1:m
        if matrix(i,1) == row(1) && matrix(i,2) == row(2)
            matrix(i,:) = [];
            break;
        end
    end

    mat_modif = matrix;
end
```

*کدها به پیوست قرار گرفته‌اند.